

null → कै पस memory नहीं

null कै जगह data या function की जगह में

null pointer Exception

Chapter 14: OOPS Architecture

- Default Values
- Relationship
 - Has A
 - Is A
 - ClassCastException
 - instanceof
- Overriding
- Overloading
- Over hiding
- Constructors
- Legal Return Types
- Coupling and Cohesion
- Inner Classes
- Garbage Collector
- Package
 - CLASSPATH
 - static import

compiler values में 46172

Default Values

1. Instance Variables

- Primitive: char by space, numerics by 0, boolean by false
- Reference: by null
- Array: by null
- if array is initialized:
 - Primitive : char by space, numerics by 0, boolean by false
 - Reference : by null

NOTE: For reference type array, new is written twice. First time for array and next time for each element.

```
class Test {  
    String a[];  
    public Test() {  
        a = new String[10];  
        for(int i = 0 ; i < a.length ; i++)  
            a[i] = new String();  
    }  
    public static void main(String args[]) {  
        Test t = new Test();  
        System.out.println(t.a.length);  
        System.out.println(t.a[0]);  
        System.out.println(t.a[0].length());  
    }  
}
```

NOTE: any data/function access with null results in NullPointerException

2. Local Variables

Local variables always initialised { Primitive, Reference, Array }

`int n;`
`SO P(n);` X

vedisoft
Software & Education Services Pvt. Ltd.

a. Primitive

- if uninitialized, it will result in compilation error
- if initialization in a conditional block (if, switch, while, for, try)
 - condition with values and true - will compile
 - condition with values and false - will not compile
 - condition with variables and true or false - will not compile

`int n;`
`if (10 > 5)`
`n = 100;`
`SO P(n);` ✓

`if (1 > 5)`
`n = 100;`
`SO P(n);` X

b. Reference

- if uninitialized, it will result in compilation error
- if initialization in a conditional block (if, switch, while, for, try)
 - condition with values and true - will compile
 - condition with values and false - will not compile
 - condition with variables and true or false - will not compile

`int n, a = 10, b = 5;`

`if (a > b)`
`n = 100;`
`SO P(n);` X

c. Array

- if uninitialized, it will result in compilation error
- if initialization in a conditional block (if, switch, while, for, try)
 - condition with values and true - will compile
 - condition with values and false - will not compile
 - condition with variables and true or false - will not compile

Relationship

- Relationship between classes.
- There are 2 types of relationships
 1. Has A
 2. Is A

1. Has A

- The relationship of a class with its instance variable classes is called Has A relationship

class Name {

String firstName;

String lastName;

}

class Person {

Name name;

}

Name Has A String

Person Has A Name

NOTE: This type of relationship can be proved

2. Is A

- The relationship of a class with itself or its base classes/interfaces is called Is A relationship

class Subject {}

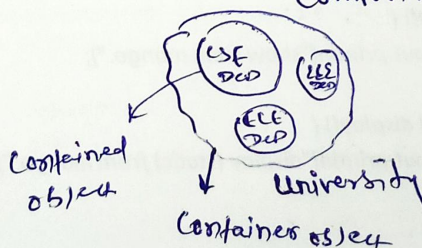
class Maths extends Subject {}

class Science extends Subject {}

① objects strongly associated.

② container object holds directly contained object

Composition



W/O existing Container object if there is no chance of existing contained object then Container and contained objects are strongly associated and

this Strong association is nothing but Composition

If we want total functionality of a class automatically then go for IS-A. If we want partial functionality then go for has-A relationship.

Subject Is A Subject
Maths Is A Subject
Maths Is A Maths
Science Is A Subject
Science Is A Science

Subject obj = new Subject();
Subject obj = new Maths();
Maths obj = new Maths();
Subject obj = new Science();
Science obj = new Science();

- To prove Is A relationship in Java "instanceof" operator is used.
- This operator should always be used while narrowing to avoid "ClassCastException"

Example:

```
class Fruits {
    public void show() {
        System.out.println("show from fruits.");
    }
    public static void display() {
        System.out.println("display (static) from fruits.");
    }
}

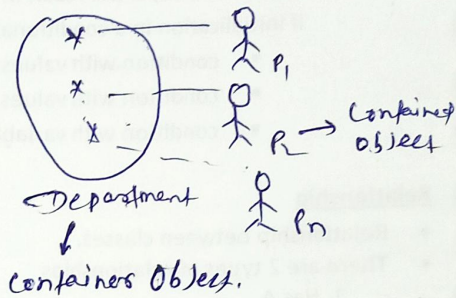
class Apple extends Fruits {
    public void show() {
        System.out.println("show from apple.");
    }
    public static void display() {
        System.out.println("display (static) from apple.");
    }
}

class Mango extends Fruits {
    public void show() {
        System.out.println("show from mango.");
    }
    public static void display() {
        System.out.println("display (static) from mango.");
    }
}

class Test {
    public static void main(String args[]) {
        Fruits o1 = new Fruits();
        o1.show();
        o1.display();

        Apple o2 = new Apple();
        o2.show();
        o2.display();
    }
}
```

Aggregation



W/o existing container object
If there is a chance of existing
contained object then
Container and contained
objects are weakly
associated and this weak
association is nothing
but aggregation.

Static function works on overriding.

↓
function declaration & function

Fruits o3 = new Apple();

o3.show();

o3.display();

// APPLE

// fruits

overriding ← member function or Normal function

overriding ← Static function

Apple o4 = new Apple();

Fruits o5 = o4;

o5.show();

// APPLE

o5.display();

// fruits.

Apple o6 = new Apple();

Fruits o7 = o6;

if (o7 instanceof Mango) {

Mango o8 = (Mango) o7;

o8.show();

o8.display();

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

Example:

interface Ia {

void a();

}

interface Ib {

void b();

}

interface Ic extends Ia, Ib {

void c();

}

abstract class Cc implements Ic {

public void a() {

System.out.println("a.");

}

}

class DerivedCc extends Cc {

public void b() {

System.out.println("b.");

}

public void c() {

System.out.println("c.");

}

public void d() {

System.out.println("d.");

}

}

}

}

}

}

}

}

}

अपने के लिए लगाने
समको लगाने के बाद Exception नहीं आयेगा

class static नहीं होता है।

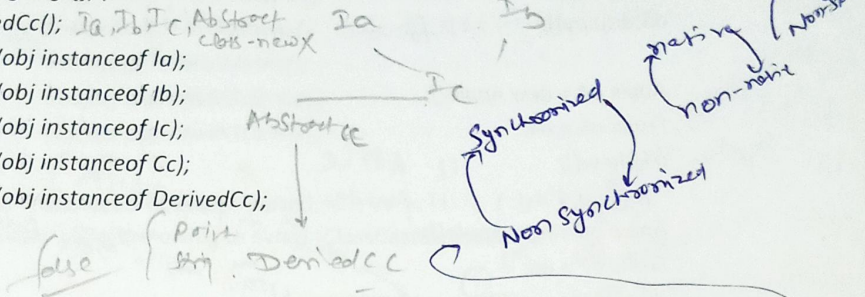
function static होता है।

Compilation Error नहीं आता है क्योंकि दोनों fruits का derive class है।

abstract class है बिना definition निश्चयन में आता

Java C - Source 1.4 Name Java

```
class Test {
    public static void main(String args[]) {
        la obj = new DerivedCc();
        System.out.println(obj instanceof la);
        System.out.println(obj instanceof lb);
        System.out.println(obj instanceof lc);
        System.out.println(obj instanceof Cc);
        System.out.println(obj instanceof DerivedCc);
    }
}
```



Overriding: Defining function again with same name, same arguments, base and derive class

Rules (Summary):

1. Access Specifier equal to or greater than
2. static
3. final
4. Return type : same, co-variant (JDK5) (base is static & Derived is object)
5. Exception: Eliminate, reduce, narrow, unchecked can be introduced
6. Above 5 rules works with functions that are inherited

Rules (Detailed):

- You cannot override a final method
- You cannot override a static method to make it non-static
- The overriding method and the overridden method must have the same return type. Java 6.0 allows co-variant return types
- The number and types of parameters in the overriding and overridden should be same, however name can be different
- If the overriding method has a throws clause in its declaration, then the overridden method can also have throws clause (elimination is possible, derive classes can be used)
- You cannot override a method that is not inherited
- You cannot override a method to make it is less accessible, but vice versa is true

```
import java.io.IOException;
class Abc {
    private static int convert(String s) {
        return Integer.parseInt(s);
    }
    public Object getMessage() {
        return "Hello World";
    }
    public static void main(String args[]) {
        System.out.println("Abc.");
    }
}
```

method overridden for Derived

Abstract class is final and it is final

Abstract class - new

Abstract

DerivedCc

Synchronized

Non Synchronized

final

Non-final

abstract

non-abstract

abstract

non-abstract

Child method

Final / Non-final

non-abstract

abstract

co-variant

in primitive return type must be same

Number and its child class

Number

String

double

Object

String

String Buffer

Number / Integer / Float

Object

Child method return type


```

class Xyz extends Abc {
    public static int convert(String s) throws IOException {
        return Integer.parseInt(s);
    }
    public String getMessage() {
        return "Hello World";
    }
    public static void main(String args[]) {
        System.out.println("Xyz.");
    }
}

```

Overloading: Defining function again with same name, different arguments in the same class or base and derive class

Rules (Summary):

1. At-least 1 argument should be different.

Rules (Detailed):

- You can change the argument list of an overloaded method
- You can change the return type (but at-least one argument should be changed) and access modifier of the overloaded method
- You can declare the new checked exceptions in the overloaded method
- You can overload a method in the same or in the subclass

Example:

```

class Abc {
    public String getMessage(char c) {
        return "Hello World";
    }
    public Object getMessage() {
        return "Hello World";
    }
    public static void main(String args[]) {
        System.out.println("Abc.");
    }
}

```

b. `public int add(int a, int b, int c) {}`
`public float add(int a, int b, int c) {}`
 ↓
 this is not overloaded function

Comparing Overloading with Overriding

Point of Difference	Overloading	Overriding
Arguments	Change number/type/sequence	Cannot Change number/type/sequence
Access Modifiers	Change → <i>only 1 method</i>	Same/Increase Access
Return Type	Change*	Cannot Change / Covariant

static function call karta hai ki stock add karta hai
 constructor ko memory me karta hai object ko memory me

Exceptions

Change

You can reduce, narrow or eliminate.
 No new checked Exceptions can be added.
 Unchecked Exceptions can be added.

Constructors Rules

1. If no constructors are written, Java adds a default constructor that initializes numerics by 0, boolean by false, character by space and reference type by null.
2. If we provide constructors, Java does not add the default constructor.
3. Whenever we create an object of derive class, base class default constructor is automatically called in the order base to derive.
4. If no default constructor is present in base class and parameterized constructor is present. Then creating an object of derive class will result in compilation error. This error can be remove by using super and calling a parameterized constructor
5. It is possible to call constructors of same class from other constructors using this(<args>)

Example:

```
class Abc {
    int a;
    public Abc() {
        System.out.println("Default Abc.");
    }
    public Abc(int a) {
        this.a = a;
        System.out.println("Parameterized Abc.");
    }
}

class Xyz extends Abc {
    int a;
    public Xyz() {
        super(10);
        System.out.println("Default Xyz.");
    }
    public Xyz(int a) {
        super(10);
        this.a = a;
        System.out.println("Parameterized Xyz.");
    }
}

class Test {
    public static void main(String args[]) {
        Xyz x = new Xyz();
    }
}
```

Example:

```
class Abc {
```



```

public Abc() {
    System.out.println("This is Default constructor of Abc.");
}

public Abc(int a) {
    this();
    System.out.println("This is Para. constructor of Abc.");
}

class Xyz extends Abc {
    public Xyz() {
        this(10);
        System.out.println("This is Default constructor of Xyz.");
    }

    public Xyz(int a) {
        super(10);
        System.out.println("This is Para. constructor of Xyz.");
    }
}

class Test {
    public static void main(String args[]) {
        Xyz a = new Xyz();
    }
}

```

Legal Return Types

function mein kahi kahi return type likhna hai

```

class Test {
    //incase return type is void, no value can be returned, only return will do
    public void sayMessage() {
        System.out.println("Hello World");
        return;
    }

    // you can any primitive type from a function
    public int getNumber() {
        return 10;
    }

    // primitive -- implicit type casting
    public int getAnotherNumber() {
        return 'C';
    }

    // you can return any reference type from a function
    public String getMessage() {

```



```

        String str = new String("Hello World");
        return str;
    }

    // for any reference type null value can be returned
    public String getAnotherMessage() {
        String str = new String("Hello World");
        return null;
    }

    // you can return an array from a function
    public int[] getNumbers() {
        int num[] = {10, 20, 30, 40, 50};
        return num;
    }

    // you can return null for an array as array is a reference type
    public int[] getNumbers1() {
        return null;
    }

    // implicit type casting - reference
    public Object getNumbers() {
        int num[] = {10, 20, 30, 40, 50};
        return num;
    }
}

```

Coupling and Cohesion

Coupling: Coupling is between 2 classes

Types of Coupling

1. Loose Coupling: Changes in one class does not very much affect another class
2. Tight Coupling: Changes in one class very much affects another class

Cohesion

- Cohesion is within a class
- How well a class is organized/structured

Good programming practice says: "**Classes should be loosely coupled and highly cohesive**"

Inner classes

I. inner class

1. local inner class
 - a. block local inner class
 - b. method local inner class

local किन बार ही Access नही होता है /
local के लिए कोई Accessor नही होता है /

2. anonymous inner class

3. member inner class*

II. static inner class*

* Object can be created within outer class or outside outer class

Block Local Inner Class

- These classes are created within a block like static {}, anonymous {}
- The object of this class can be created only within the block
- It cannot contain static data and static functions
- It cannot be marked as private, public
- Naming: Outer\$1Inner.class

Outer class कि होगा

Example: static block

```
class Test {  
    static {  
        class Abc {  
            public void show() {  
                System.out.println("show from abc.");  
            }  
        }  
        Abc obj = new Abc();  
        obj.show();  
    }  
    public static void main(String args[]) {  
        Test o1 = new Test();  
        Test o2 = new Test();  
        Test o3 = new Test();  
        Test o4 = new Test();  
    }  
}
```

static block इस तरह चलाता है /

static block class के लिए चलाता है /

इस हिस्से show from Abc मिलेगी

anonymous object
object के लिए चलाता है

Example : anonymous block

```
class Test {  
    {  
        class Abc {  
            public void show() {  
                System.out.println("show from abc.");  
            }  
        }  
        Abc obj = new Abc();  
        obj.show();  
    }  
    public static void main(String args[]) {  
        Test o1 = new Test();  
        Test o2 = new Test();  
    }  
}
```



```
Test o3 = new Test();
Test o4 = new Test();
```

```
}
```

Method local inner class

- These classes are created within a method
- The object of this class can be created only within the method
- It cannot contain static data and static functions
- It cannot be marked as private, public
- Naming: Outer\$1Inner.class

outer class में बनाया

Example:

```
class Test {
```

```
    public void check() {
```

```
        class Abc {
```

```
            public void show() {
```

```
                System.out.println("show from abc.");
```

```
            }
```

```
        }
```

```
        Abc obj = new Abc();
```

```
        obj.show();
```

```
    }
```

```
    public static void main(String args[]) {
```

```
        Test o1 = new Test();
```

```
        o1.check();
```

```
    }
```

```
}
```

*जितना बार object बनाके
करकेगा उतना बार show चलेगा।
check जितना बार call करेगी उतना बार
show call होगा चाहे object same हो या
different.*

Anonymous inner class

- These classes are created to override a method for an object.
- New methods can be added but cannot be called from outside the class.
- It cannot contain static data and static functions.
- Naming: Outer\$1.class, Outer\$2.class

आम static inner class में होता

Example:

```
class Abc {
```

```
    public void show() {
```

```
        System.out.println("show from abc.");
```

```
    }
```

```
    public void display() {
```

```
        System.out.println("display from abc.");
```

```
    }
```

```
}
```

```
class Test {
```


new Point(). Show Date()

class 3007070707

point 20 21 21

object 07070707 3070



vedisoft®
Software & Education Services Pvt. Ltd.

d's * class

```
public static void main(String args[]) {
```

```
    Abc obj = new Abc();
```

```
    obj.show();
```

```
    obj.display();
```

obj 1 ke liye Anonymous class bana rha hai

```
    Abc obj1 = new Abc() {
```

```
        public void display() {
```

```
            System.out.println("overridden display.");
```

```
        }
```

```
    };
```

```
    obj1.show();
```

```
    obj1.display();
```

```
    new Abc() {
```

```
        public void display() {
```

```
            System.out.println("another overridden display.");
```

```
        }
```

```
    }.display();
```

```
}
```

```
}
```

Member Inner Class

- A class within a class is called member inner class
- Object can be created within outer class or outside outer class
- It cannot contain static data and static function
- It can be marked as private, public
- Naming: Outer\$Inner.class
- To create an object of member inner class outside outer class, outer class object is required

```
Outer o = new Outer();
```

```
Outer.Inner i = o.new Inner();
```

```
OR
```

```
Outer.Inner i = new Outer().new Inner();
```

{ class ke andar public hai default
chalta hai - oops Lecture 4 }

Example:

```
class Abc {
```

```
    class Xyz {
```

```
        public void show() {
```

```
            System.out.println("show from Xyz");
```

```
        }
```

```
    }
```

```
    public void check() {
```

```
        //creating an object of inner class within outer class
```

```
        Xyz x = new Xyz();
```

```
        x.show();
```

```
    }
```

```
}
```

outer class ke andar inner class ka object banate hai


```
class Test {
    public static void main(String args[]) {
        Abc a = new Abc();
        a.check();

        //creating an object of inner class outside outer class
        Abc a1 = new Abc();
        Abc.Xyz obj = a1.new Xyz();
        obj.show();

        //creating an object of inner class outside outer class
        Abc.Xyz obj1 = new Abc().new Xyz();
        obj1.show();
    }
}
```

Static inner class

- An inner created explicitly using static keyword is called static inner class
- Object can be created within outer class or outside outer class
- It can contain static data and static function
- It can be marked as private, public
- Naming : Outer\$Inner.class
- To create an object of static inner class outside outer class, outer class object is not required

```
Outer.Inner i = new Outer.Inner();
```

Example:

```
class Abc {
    static class Xyz {
        public void show() {
            System.out.println("show from Xyz");
        }
        public static void display() {
            System.out.println("static display from Xyz");
        }
    }

    public void check() {
        //creating an object of static inner class within outer class
        Xyz x = new Xyz();
        x.show();
    }
}

class Test {
    public static void main(String args[]) {
        Abc a = new Abc();
        a.check();
        //creating an object of inner class outside outer class
    }
}
```



```

    Abc.Xyz obj = new Abc.Xyz();
    obj.show();
    Abc.Xyz.display();
}

```

श्री - श्री २ पर चलता

जिसका scope खत्म हो जाता है उसके लिए चलता है।

Garbage Collector

- Java does not provide destructors, rather than destructors, Java provides a utility called as "Garbage Collector"
- This utility executes periodically and releases the unused memory (memory whose scope is over, or initialized by null)
- If you want your statements to be executed along with "Garbage Collector" you can provide them within protected void finalize()
- "Garbage Collector" can also be invoked by programmer by calling System.gc();

```

class Test {
    protected int a;
    public Test(int a) {
        this.a = a;
    }
    public static void check() {
        Test t1 = new Test(40);
        Test t2 = new Test(50);
    }
    public void nullFunction(Test t) {
        t = null;
    }
    protected void finalize() {
        System.out.println("Garbage Collector : " + a);
    }
    public static void main(String args[]) {
        Test t1 = new Test(10);
        Test t2 = new Test(20);
        Test t3 = new Test(30);
        t3 = null; // Garbage collector 30
        t1.nullFunction(t2);
        check();
        Test t4 = new Test(60);
        Test t5 = new Test(70);
        Test t6 = new Test(80);
        t4 = t5;
        t5 = t6;
        t6 = t4;
        t4 = null;
        System.gc();
        try { Thread.sleep(200); } catch (Exception e) {}
    }
}

```

① deprecated

variable address

t2	1024
t1	

20
1024

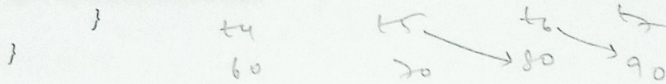
60 70 80
1012 1014 1048
91
1008

t4	1012
t5	1014
t6	1048
t3	1008

मल्टीपल ऑब्जेक्ट्स हो सकते हैं

debaicope - पूरा नाम तब तक है जब तक चल रहा है

Static - 5
Arithmetic 4



static import

- It is a feature of JDK5
- It is used to import static data and static functions of a class
- Variants

import static package.ClassName.*;

//all static data and static functions from specified class

import static package.ClassName.staticData;

//specific static data from specified class

import static package.ClassName.staticFunction;

//specific static function from specified class

//import static java.lang.Math.*;

import static java.lang.Math.PI;

import static java.lang.Math.pow;

class Test {

public static void main(String args[]) {

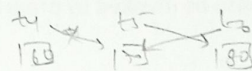
System.out.println(PI);

System.out.println(pow(10,2));

}

}

original question



Packages - Example

Level - I

1. mypack (Abc, MainProg)

a. mysubpack1 (Abc, Efg)

i. mysubsubpack1 (Abc, Pqr)

ii. mysubsubpack2 (Abc, Xyz)

b. mysubpack2 (Abc, Lmn)

Level - II

C:\data a. anotherpack (Abc)

F:\ b. somepack (Abc)

Rules:

1. Packaged class should always start with package statement

2. Packaged class if needs to be accessed from outside the package, it should be declared public

package mypack.mysubpack1.mysubsubpack2;

public class Abc {

public Abc() {

System.out.println("Abc from mypack.mysubpack1.mysubsubpack2.");

}

}

package mypack.mysubpack1.mysubsubpack2;

public class Xyz {


```
public Xyz() {
    System.out.println("This is Xyz.");
}
```

3. Packaged class needs to be compiled and interpreted from outside the package.

D:\Data>javac mypack\MainProg.java

D:\Data>java mypack.MainProg

4. Classes from same package need not to be imported

5. Specific import suppresses all other import

import mypack.mysubpack1.Abc;

6. In case of ambiguity inline import is used.

mypack.mysubpack2.Abc obj2 = new mypack.mysubpack2.Abc();

7. In case of no ambiguity, no suppression, normal import is used

import mypack.mysubpack2.*;

8. To import classes from other drives/directories, CLASSPATH should be set

D:\Data>set CLASSPATH=.;C:\Data\F:\

OR

D:\Data>javac -classpath .;C:\Data\F:\ mypack\MainProg.java

D:\Data>java -classpath .;C:\Data\F:\ mypack.MainProg

NOTE: Only packaged classes can be imported

package mypack;

import mypack.mysubpack1.Abc;

import mypack.mysubpack1.*;

import mypack.mysubpack2.*;

import mypack.mysubpack1.mysubsubpack1.*;

import mypack.mysubpack1.mysubsubpack2.*;

public class MainProg {

public static void main(String args[]) {

Abc obj = new Abc();

mypack.Abc obj1 = new mypack.Abc();

mypack.mysubpack2.Abc obj2 = new mypack.mysubpack2.Abc();

mypack.mysubpack1.mysubsubpack1.Abc obj3 = new mypack.mysubpack1.mysubsubpack1.Abc();

mypack.mysubpack1.mysubsubpack2.Abc obj4 = new mypack.mysubpack1.mysubsubpack2.Abc();

Efg obj5 = new Efg();

Lmn obj6 = new Lmn();

Pqr obj7 = new Pqr();

Xyz obj8 = new Xyz();

anotherpack.Abc obj9 = new anotherpack.Abc();

somepack.Abc obj10 = new somepack.Abc();

}

}

single line

import class

package

import class

package

import class

package

import class

Throwable class act as a root for exception hierarchy.

Chapter 10: Exception Handling

try/catch

- try
- catch
- finally
- throw
- throws

An unexpected, unwanted event that disrupts normal flow of the program is called exception

Types of Errors

1. Compile Time/Syntax Errors

Examples: missing ; , misspelling, missing " , using variable without declaration etc

2. Runtime Errors

Examples: division by Zero, Invalid Conversion, Accessing invalid index of an array.

- Runtime error results in an Exception
- Whenever Exception occurs Java creates an Object of "Exception" or its derive class
- Exception leads to abnormal termination of program giving some garbage on screen.
- Handling means making the user aware with problem
- For Exception Handling Java provides:
 - Constructs: try, catch, finally
 - Statements: throw, throws

try

- All statements that may result in an error are written within try.
- A method can have nested or multiple try blocks.

catch

- Every "try" should be followed by at least 1 catch block.
- Multiple catch blocks works like cases in switch only one will be executed depending on Exception generated.
- catch receives an argument of Exception or its derive class.
- catch is conditional

finally

- finally can be used as a substitute or along with catch.
- There can be only 1 finally.
- finally does not receive any argument.
- finally is unconditional.

throw

- It is used to generate an Exception

throws

- It is used to forward an Exception

जिस method में Exception आता है वो Exception का object बनाके, जो throw, location पर जाऊँगा, जहाँ उसको handle करना है। उस location पर जहाँ handle करना है या नहीं, नहीं होने पर abnormal हो जाता है। उस method को stack में ही हटाता है उसको बाद में call नही किया जाये। उसके पास आता है जो process चल रहा है तब तक कि JVM running है उसके पास नहीं पहुँच जाये उसके बाद JVM running में handle नहीं मिलेगा। क्योंकि उसको ही call करना है और उसके बाद default exception handler को जामा दे कि कैसे वह फिर उसे handle करे।

never handle two exception at a time

Example:

```
class Exception1 {  
    public static void main(String args[]) {  
        int a = 10, b = 0, c = 0;  
        c = a / b;  
        System.out.println("Result is : " + c);  
    }  
}
```

Note: Division By Zero results in "**ArithmeticException**"

Example:

```
class Exception2 {  
    public static void main(String args[]) {  
        int a = 10, b = 0, c = 0;  
        try{  
            c = a / b;  
        }  
        catch(ArithmeticException ae){  
            System.out.println("Division by Zero");  
            System.out.println(ae.getMessage());  
            ae.printStackTrace();  
        }  
        System.out.println("Result is : " + c);  
    }  
}
```

exception class
on function

String class as
exception

line numbers and message

Write a program that accepts a number as command line argument and display its square.

D:\Module1>java Exception3 5

Square : 25

D:\Module1>java Exception3 five

Whenever String to number conversion fails, java generates "**NumberFormatException**"

D:\Module1>java Exception3

Whenever we access invalid index of an array it results in "**ArrayIndexOutOfBoundsException**"

```
class Exception3 {  
    public static void main(String args[]) {  
        int num=0;  
        try {  
            num = Integer.parseInt(args[0]);  
        }  
        catch(ArrayIndexOutOfBoundsException ae) {
```

no valid value
index error

String to number conversion
Java object error

finally used for closed the Resource.
Try and Catch used for handled the exception.

```

        System.out.println("Invalid Index");
    }
    catch(NumberFormatException ne) {
        System.out.println("Invalid Format");
    }
    catch(Exception e) {
        System.out.println("Some Error"); / e.printStackTrace();
    }
    System.out.println("Square: " + Math.pow(num,2));
}

```

prescribed method

Example:

```

class Exception4 {
    public static void main(String args[]) {
        int num=0;
        try {
            num = Integer.parseInt(args[0]);
        }
        finally {
            System.out.println("Square is : " + Math.pow(num,2));
        }
    }
}

```

Exception handling part finally run

finally run, if error

In any situation safe method run

Example:

```

class BigException extends Exception { //BigException.java
    protected String msg;
    public BigException(){}
    public BigException(String msg) {
        this.msg = msg;
    }
    public String getMsg() {
        return msg;
    }
    public void setMsg(String msg) {
        this.msg = msg;
    }
}

```

```

class Exception5 { //Exception5.java
    public static void main(String args[]) {
        int num=0;
        try {

```



```

num = Integer.parseInt(args[0]);
if(num>100) {
    BigException b = new BigException("Number too big");
    num=0;
    throw b;
}
}
catch(NumberFormatException ne) {
    System.out.println("Invalid Format");
}
catch(ArrayIndexOutOfBoundsException ae) {
    System.out.println("Invalid Index");
}
catch(BigException be) {
    System.out.println(be.getMessage());
}
System.out.println("Square is : " + Math.pow(num,2));
}
}

```

Example:

```

class Exception6 {
/*

```

```

    public static int convert(String s) {
        int n = 0;
        try {
            n = Integer.parseInt(s);
        }
        catch(NumberFormatException ne) {
            System.out.println("INVALID FORMAT");
        }
        return n;
    }
}

```

```

    public static int convert(String s) throws NumberFormatException {
        int n = 0;
        n = Integer.parseInt(s);
        return n;
    }
}

```

```

    public static void main(String args[]) {
        int num=0;
        try {

```

catch ko forward karate
 is se exception forward
 kar sakte hai by comma
 put


```

num = Exception6.convert(args[0]);
}
catch(NumberFormatException ne) {
    System.out.println("Invalid Format");
}
catch(ArrayIndexOutOfBoundsException ae) {
    System.out.println("Invalid Index");
}
catch(Exception e) {
    e.printStackTrace();
}
System.out.println("Square is : " + Math.pow(num,2));
}
}

```

Exceptions can be classified in 2 ways:

Classification 1:

1. Pre Defined Exceptions
2. User Defined Exceptions

Classification 2:

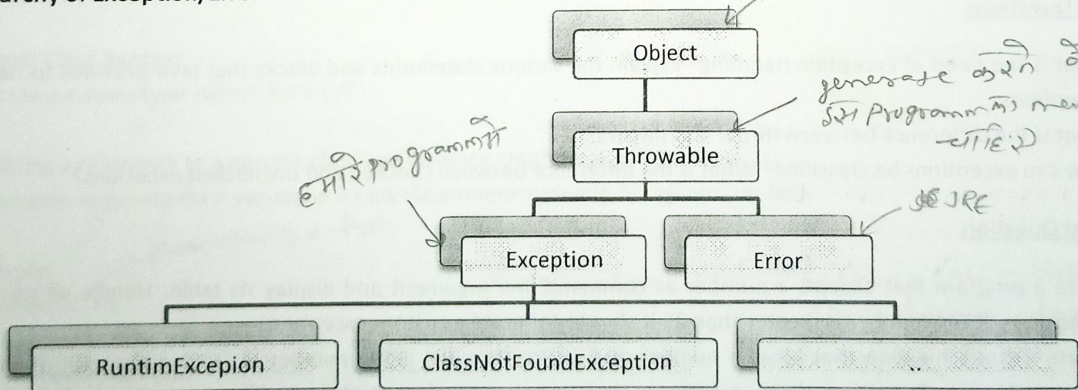
1. Unchecked Exceptions/Runtime Exceptions

- These exceptions are optional to be handled
- Generally, these except comes from java.lang package
- They are the derive classes of **RuntimeException**

2. Checked Exceptions

- These exceptions should be compulsorily handled
- These exceptions does not comes from java.lang package
- They are not the derive classes of **RuntimeException**

Hierarchy of Exception/Error



Derive classes of RuntimeException

- ArithmeticException
- IndexOutOfBoundsException
 - ArrayIndexOutOfBoundsException
 - StringIndexOutOfBoundsException
- ArrayStoreException
- ClassCastException
- IllegalArgumentException
 - NumberFormatException
 - IllegalThreadStateException
- IllegalMonitorStateException
- IllegalStateException
- NegativeArraySizeException
- NullPointerException
- SecurityException

① यदि base class उपर लिखेंगे Derived class exception से पहले catch से तो compiler एरर देगा।

② try में एरर आने के बाद वहाँ से Remaining code नहीं चलेगी केवल try post-catch के जिसे वाला सारा चलेगा.

Error

- LinkageError
 - NoClassDefFoundError
 - ClassFormatError
 - ..
- VirtualMachineError
 - InternalError
 - OutOfMemoryError
 - ..

Note: In case of multiple catch blocks, the sequence of catch should be from derive to base.

Assignment

Theory Questions

1. What is the need of Exception Handling? Explain the various statements and blocks that Java provides to handle exceptions.
2. What is the difference between throw and throws?
3. How can exceptions be classified? What is the difference between checked and unchecked exceptions?

Practical Question

1. Write a program that accepts a number as command line argument and display its table. Handle all possible exceptions. If the number is greater than 25 it should generate a custom exception.
2. Create a static function that takes a number and return its cube, if the number is greater than 100 it should generate **NoCubeException**. Write a program that accepts a number as command line argument and display its cube. Cube should be calculated using the static function created.
3. In the above programs try to use finally

Chapter 11: Generics

use reference type
not primitive type

- What is Generics?
- Difference between Generics and Object type.
- Implementing Generics.
- Generic Bounded Types
- Generic Methods
- Generic Constructor
- Generic Interface
- Wildcard
- Wildcard with upper bound
- Generic restrictions

Generics

- Introduced by JDK 5, generics changed Java in two important ways.
 - It added a new syntactical element to the language.
 - It caused changes to many of the classes and methods in the core API. (Library ko generic banana)
- With generics, you can define an algorithm once, independently of any specific type of data, and then apply that algorithm to a wide variety of data types.

What is Generics?

- Generics mean parameterized types.
- Parameterized types enable you to create classes, interfaces, and methods in which the type of data upon which they operate is specified as a parameter.
- A class, interface, or method that operates on a parameterized type is called generic class, generic interface or generic method.

generics
↓
compilation error
↓
runtime error

Difference between Generics and Object type

- With Generics, type casting is implicit. But, with Object type type casting is required.

NOTE: Generics works only with Object type, primitive data types cannot be used.

Generic Class Syntax:

```
class class-name<type-param-list> { // ...
```

Declaring a reference to a generic class and instance creation:

```
class-name<type-arg-list> var-name = new class-name<type-arg-list>(cons-arg-list);
```

Example:

```
class Gen1<T> {
    T a;
    public Gen1() {
    }
    public Gen1(T a) {
        this.a = a;
    }
}
```

parameterized type

Generics
↓
Generic function
" class
" interface

function-call - data type
class-object - data type
interface - implement - data type

हम class को base class object है।
 Fully Qualified name package के साथ class का नाम

```

    }
    public void setA(T a) {
        this.a = a;
    }
    public T getA() {
        return a;
    }
    public void showType() {
        System.out.println("Type of T is : " + a.getClass().getName());
    }
    public static void main(String args[]) {
        Gen1<Integer> obj = new Gen1<Integer>();
        obj.setA(10);
        System.out.println("Value : " + obj.getA());
        obj.showType();

        Gen1<String> obj1 = new Gen1<String>();
        obj1.setA("Welcome");
        System.out.println("Value : " + obj1.getA());
        obj1.showType();

        // Gen1<int> obj2 = new Gen1<int>();
    }
}
    
```

object की class बताता है



class का नाम बताता है।

सामान्य होना पर निरूपित है।

compulsary

primitive तो STM होता है

Example:

```

class Gen2<T, V> {
    T a;
    V b;
    public Gen2() {
    }
    public Gen2(T a, V b) {
        this.a = a;
        this.b = b;
    }
    public void setA(T a) {
        this.a = a;
    }
    public T getA() {
        return a;
    }
    public void setB(V b) {
        this.b = b;
    }
}
    
```



```

public V getB() {
    return b;
}

public void showType() {
    System.out.println("Type of T is : " + a.getClass().getName());
    System.out.println("Type of V is : " + b.getClass().getName());
}

public static void main(String args[]) {
    Gen2<Integer, String> obj = new Gen2<Integer, String>();
    obj.setA(10);
    obj.setB("Welcome");
    System.out.println("Value : " + obj.getA());
    System.out.println("Value : " + obj.getB());
    obj.showType();
}
}

```

Example: (Bounded Types)

bound करने के लिए जिनका object हो जाता है

```

class Gen3<T extends Number> {
    T num[];
    public Gen3(T num[]) {
        this.num = num;
    }

    public double average() {
        double sum = 0;
        for (int i = 0; i < num.length; i++) {
            sum += num[i].doubleValue();
        }
        return sum / num.length;
    }

    public static void main(String args[]) {
        Integer num[] = { 1, 2, 3, 4, 5, 6 };
        Gen3<Integer> obj = new Gen3<Integer>(num);
        System.out.println(obj.average());

        Double num1[] = { 1.1, 2.2, 3.3, 4.4, 5.4, 6.6 };
        Gen3<Double> obj1 = new Gen3<Double>(num1);
        System.out.println(obj1.average());
        /*
        * String str[] = {"Sunday", "Monday", "Tuesday"}; Gen3<String> obj3 = new
        * Gen3<String>(str);
        */
    }
}

```

होता बना रहे है

State function - by class
by object
includes as it's.



vedisoft
Software & Education Services Pvt. Ltd.

Example: (Generic Method)

```
class Gen4 {
    public static <T, V extends T> boolean isln(T num[], V num1) {
        for (int i = 0; i < num.length; i++) {
            if (num[i].equals(num1))
                return true;
        }
        return false;
    }
}
```

T के अंदर V को search कर रहे हैं

String के अंदर String को search कर रहे हैं

Generic function
में Return
Type है पता

```
public static void main(String args[]) {
    String str[] = { "One", "Two", "Three", "Five" };
    Integer num[] = { 1, 2, 3, 5 };
    if (!isln(str, "Four"))
        System.out.println("Four not present");
    if (isln(str, "Three"))
        System.out.println("Three present");
    if (!isln(num, 4))
        System.out.println("Four not present");
    if (isln(num, 3))
        System.out.println("Three present");
    if (isln(num, "3"))
        System.out.println("Three present");
}
```

String नहीं लिख सकते हैं।

Example: (Generic Constructors)

```
class Gen5 {
    double data;
    public <T extends Number> Gen5(T obj) {
        data = obj.doubleValue();
    }
    public static void main(String args[]) {
        Gen5 obj1 = new Gen5(10);
        Gen5 obj2 = new Gen5(10l);
        Gen5 obj3 = new Gen5(10.5);
        Gen5 obj4 = new Gen5(10.6f);
    }
}
```

द्वारा करने के लिए extend लगाने हैं

Number को Double में बदल रहे हैं

Example: (Generic Interfaces)

```
interface MyInterface<T> {
    void sayMessage(T t);
}
```



```
class MyClass implements MyInterface<String> {
    public void sayMessage(String t) {
        System.out.println(t);
    }

    public static void main(String args[]) {
        MyClass obj = new MyClass();
        obj.sayMessage("Hello World");
    }
}
```

Example: Wildcard

```
class Stats<T extends Number> {
```

```
    T[] nums;
```

```
    Stats(T[] p) {
```

```
        this.nums = p;
```

```
    }
```

```
    double average() {
```

```
        double sum = 0.0;
```

```
        for (int i = 0; i < nums.length; i++)
```

```
            sum += nums[i].doubleValue();
```

```
        return sum / nums.length;
```

```
    }
```

```
    boolean sameAvg(Stats<?> ob) {
```

```
        if (average() == ob.average())
```

```
            return true;
```

```
        return false;
```

```
    }
```

```
}
```

Number 3773 3773
derive type

arg on datatype double

wildcard (integer, float, double...)
for comp

wrong context

argument

```
class WildcardDemo {
```

```
    public static void main(String args[]) {
```

```
        Integer inums[] = { 1, 2, 3, 4, 5 };
```

```
        Stats<Integer> iob = new Stats<Integer>(inums);
```

```
        double v = iob.average();
```

```
        System.out.println("iob average is " + v);
```

```
        Double dnums[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };
```

```
        Stats<Double> dob = new Stats<Double>(dnums);
```

```
        double w = dob.average();
```

```
        System.out.println("dob average is " + w);
```

```
        Float fnums[] = { 1.0F, 2.0F, 3.0F, 4.0F, 5.0F };
```

```
        Stats<Float> fob = new Stats<Float>(fnums);
```

```
        double x = fob.average();
```

```
        System.out.println("fob average is " + x);
```


Constant
content *Argument*

```

System.out.print("Averages of iob and dob ");
if (iob.sameAvg(dob))
    System.out.println("are the same.");
else
    System.out.println("differ.");
System.out.print("Averages of iob and fob ");
if (iob.sameAvg(fob))
    System.out.println("are the same.");
else
    System.out.println("differ.");
    }
}

```

Example: WildCard(Upper Bound)

```

class TwoD {
    int x, y;
    TwoD(int a, int b) {
        x = a;
        y = b;
    }
}

class ThreeD extends TwoD {
    int z;
    ThreeD(int a, int b, int c) {
        super(a, b);
        z = c;
    }
}

class FourD extends ThreeD {
    int t;
    FourD(int a, int b, int c, int d) {
        super(a, b, c);
        t = d;
    }
}

class Coords<T> extends TwoD {
    T[] coords;
    Coords(T[] o) {
        coords = o;
    }
}

class BoundedWildcard {
    static void showXY(Coords<?> c) {
        System.out.println("X Y Coordinates:");
    }
}

```

2D, 3D, 4D
→ into derive class


```

for (int i = 0; i < c.coords.length; i++)
    System.out.println(c.coords[i].x + " " + c.coords[i].y);
System.out.println();
}
static void showXYZ(Coords<? extends ThreeD> c) {
    System.out.println("X Y Z Coordinates:");
    for (int i = 0; i < c.coords.length; i++)
        System.out.println(c.coords[i].x + " " + c.coords[i].y + " " + c.coords[i].z);
    System.out.println();
}
static void showAll(Coords<? extends FourD> c) {
    System.out.println("X Y Z T Coordinates:");
    for (int i = 0; i < c.coords.length; i++)
        System.out.println(c.coords[i].x + " " + c.coords[i].y + " " + c.coords[i].z + " " + c.coords[i].t);
    System.out.println();
}
}
public static void main(String args[]) {
    TwoD td[] = { new TwoD(0, 0), new TwoD(7, 9), new TwoD(18, 4), new TwoD(-1, -23) };
    Coords<TwoD> tdlocs = new Coords<TwoD>(td);
    System.out.println("Contents of tdlocs.");
    showXY(tdlocs); // OK, is a TwoD
    // showXYZ(tdlocs); // Error, not a ThreeD
    // showAll(tdlocs); // Error, not a FourD
    // Now, create some FourD objects.
    FourD fd[] = { new FourD(1, 2, 3, 4), new FourD(6, 8, 14, 8), new FourD(22, 9, 4, 9),
                    new FourD(3, -2, -23, 17) };
    Coords<FourD> fdlocs = new Coords<FourD>(fd);
    System.out.println("Contents of fdlocs.");
    // These are all OK.
    showXY(fdlocs);
    showXYZ(fdlocs);
    showAll(fdlocs);
}
}

```

← 3D को 3D में 4D

Bounded wildcard

→ 4D से 4D

Generic Restrictions

- Type parameters cannot be instantiated.
- No static member can use a type parameter declared by the enclosing class.
- You cannot instantiate an array whose element type is a type parameter.
- You cannot create an array of type-specific generic references.
- A generic class cannot extend Throwable.

$T = \text{new } T \times$ $\text{GenT} = \text{new } T$ ✓
↑
class के new मिलने है

② class Gen1<T> {
 Ta
 public static.... }

cannot access T

}